Edge-Guided Residual Depth Correction for Monocular Depth Estimation using MiDaS

Sai Sudarshan,Sai Chandra Raju,Bhagya Shree

April 23, 2026

Abstract

Monocular depth estimation is a fundamental computer vision problem that predicts dense scene depth from a single RGB image. It is useful in robotics, augmented reality, human-computer interaction, scene understanding, and visual reasoning. In this project, MiDaS is used as a strong baseline for monocular depth estimation because of its robust zero-shot generalization across diverse scenes. However, the raw output of MiDaS still exhibits common local artifacts such as blurred boundaries, depth bleeding across foreground and background regions, and noise in low-texture areas.

To improve the depth output beyond simple post-processing, this project develops an edge-guided residual depth correction pipeline. The system first generates a baseline depth map using MiDaS, then constructs an edge-aware refinement target using guided filtering, and finally trains a small learnable correction network that predicts a residual depth adjustment. The final output is therefore not only smoothed, but actually enhanced through a learned boundary-aware correction module. The proposed pipeline combines deep depth priors, classical image-guided refinement, and residual learning into a unified framework.

The final system produces raw MiDaS depth maps, refined guided-filter depth maps, and enhanced depth maps generated by the residual correction network. Visual results show sharper boundaries, smoother surfaces, and better alignment of depth discontinuities with image edges while preserving the global structure predicted by MiDaS. The project demonstrates how a pretrained monocular depth model can be extended into a stronger edge-guided depth enhancement pipeline without retraining the baseline estimator from scratch.

Contents

1 Introduction **4**
- 1.1 Problem Statement . . . 4
- 1.2 Objective . . . 4
- 1.3 Contribution . . . 5

2 Background and Related Concepts **5**
- 2.1 Monocular Depth Estimation . . . 5
- 2.2 MiDaS as a Baseline Model . . . 5
- 2.3 Common Limitations of Raw Depth Maps . . . 5
- 2.4 Guided Filtering . . . 5
- 2.5 Residual Learning . . . 6

3 Baseline MiDaS Architecture **6**
3.1 High-Level Architecture . . . 6
3.2 MiDaS as a Depth Prior . . . 6

4 Proposed Edge-Guided Residual Depth Correction Framework **7**
4.1 Overall System Flow . . . 7
4.2 Classical Edge-Aware Refinement . . . 7
4.3 Learnable Residual Correction . . . 7
4.4 Why Residual Correction is Useful . . . 7

5 Mathematical Formulation **8**
5.1 Baseline Depth Prediction . . . 8
5.2 Depth Normalization . . . 8
5.3 Edge Map Extraction . . . 8
5.4 Guided Filtering Model . . . 8
5.5 Residual Depth Correction . . . 9
5.6 Loss Function . . . 9

6 Implementation Details **9**
6.1 Project File Structure . . . 9
6.2 Folder Layout . . . 10
6.3 Baseline Inference Command . . . 10
6.4 Guided Refinement Command . . . 10
6.5 Training Command . . . 10
6.6 Inference Command . . . 11
6.7 Training Hyperparameters . . . 11

7 Experimental Setup **11**
7.1 Input Images . . . 11
7.2 Training Targets . . . 11
7.3 Evaluation Style . . . 11

8 Results and Analysis **12**
8.1 Typical Observations . . . 12
8.2 Figure Gallery . . . 12
8.3 Detailed Qualitative Discussion . . . 13
8.4 Failure Cases . . . 13

9 Discussion **13**

10 What Has Been Done in This Project **14**

11 Future Work **14**

12 Conclusion **14**

A Commands Used **15**
A.1 Install Dependencies 15
A.2 MiDaS Baseline 15
A.3 Guided Refinement 15
A.4 Training the Residual Refiner 15
A.5 Running the Enhanced Model 15

B Suggested Figure Naming Convention **16**

C References **16**

1 Introduction

Depth estimation is the process of determining how far different parts of a scene are from the camera. In computer vision, this task is important because depth converts a 2D image into a more meaningful spatial representation. A reliable depth map can support 3D scene reconstruction, obstacle avoidance, robotics, virtual object insertion, augmented reality, and scene interpretation. Traditional systems such as stereo cameras or LiDAR can provide depth directly, but they require extra hardware, careful setup, or multiple synchronized views. Monocular depth estimation is attractive because it works with a single RGB image and can therefore be used in many practical settings.

The challenge in monocular depth estimation is that a single image does not contain explicit 3D structure. The depth must be inferred from appearance cues such as perspective, shading, object scale, occlusion, texture gradients, and semantic priors. Deep learning methods have significantly improved this task by learning such cues from large datasets. Among them, MiDaS has become one of the most robust and widely used models because it generalizes well across different scenes and datasets.

Even though MiDaS performs strongly as a baseline, its depth maps are not always locally perfect. The prediction may be blurred at object boundaries, may show depth bleeding between adjacent objects, and may contain noise in smooth regions such as floors, walls, roads, or the sky. These imperfections reduce the usefulness of the output when visual quality, interpretability, or local geometric detail matter. This is especially important in applications where the depth map is directly inspected by humans or used in downstream image processing.

This project extends MiDaS in a more research-oriented way. Instead of stopping at raw depth estimation or simple post-processing, the work introduces an edge-guided residual correction pipeline. The output of MiDaS is first refined using an edge-aware guided filtering step, and then that refined depth is used as a pseudo-target to train a small residual network. The final system does not merely smooth the depth map; it learns how to correct the baseline depth using structure from the original image. In that sense, the project moves from baseline inference to boundary-aware depth enhancement.

1.1 Problem Statement

The baseline depth output from MiDaS is globally meaningful but locally imperfect. The problem addressed in this project is how to improve local boundary fidelity, reduce local noise, and preserve scene geometry while keeping the strong depth prior learned by MiDaS.

1.2 Objective

The main objective is to build a depth enhancement pipeline that:

- preserves object boundaries more accurately,
- reduces noisy fluctuations in flat regions,
- improves the visual consistency of the depth map,
- and learns a residual correction on top of the pretrained depth prior.

1.3 Contribution

The contribution of this project is an edge-guided depth enhancement framework built on top of MiDaS. The project combines:

- a pretrained monocular depth estimator,
- edge-aware guided filtering,
- a learnable residual correction module,
- and a complete training/inference pipeline for enhanced depth generation.

2 Background and Related Concepts

2.1 Monocular Depth Estimation

Monocular depth estimation assigns a depth value to each pixel in a single RGB image. Since there is no second viewpoint, the model must infer depth using learned priors rather than direct geometric triangulation. This makes the task inherently ambiguous, but also extremely useful in environments where stereo or LiDAR sensing is not available.

2.2 MiDaS as a Baseline Model

MiDaS is a monocular depth estimation framework known for strong cross-dataset generalization. The model family includes several versions and backbones, such as ResNet-based, EfficientNet-based, ViT-based, Swin-based, BEiT-based, and other transformer or convolutional variants. MiDaS is especially useful as a baseline because it predicts relative depth robustly even on scenes that were not seen during training. In this project, MiDaS serves as the starting point for all refinement and enhancement.

2.3 Common Limitations of Raw Depth Maps

Although MiDaS is strong, raw depth maps often display:

- blurred depth transitions near object boundaries,
- depth bleeding across adjacent regions,
- noisy variation in smooth areas,
- and weak local structural precision.

These issues motivate the need for a refinement strategy that respects both image content and depth smoothness.

2.4 Guided Filtering

Guided filtering is an edge-preserving smoothing technique that uses a guidance image to control the output. The output is assumed to be a local linear transform of the guidance image, which helps preserve strong edges while smoothing less important variations. This makes guided filtering particularly suitable for depth refinement, because depth should remain smooth on surfaces but change sharply at real object boundaries.

2.5 Residual Learning

Residual learning is a powerful modeling idea in which a network learns to predict the correction *on top of* a baseline prediction rather than learning the entire output from scratch. In this project, the residual network predicts a depth correction map ΔD that is added to the MiDaS output. This focuses the network on the missing details and boundary corrections instead of the global depth structure.

3 Baseline MiDaS Architecture

MiDaS follows a conventional encoder-decoder design. The encoder extracts multi-scale features from the input RGB image, and the decoder transforms those features into a dense depth map. In earlier versions, the encoder was based on convolutional backbones such as ResNet. In later versions, transformer backbones such as ViT, BEiT, Swin, and Next-ViT were introduced to capture longer-range dependencies and improve the quality of dense predictions.

3.1 High-Level Architecture

At a high level, MiDaS can be summarized as:

- **Input:** a single RGB image.
- **Encoder:** a deep feature extractor that captures global and local scene information.
- **Decoder:** an upsampling and feature-fusion module that reconstructs a dense depth map.
- **Output:** a relative depth prediction with strong scene generalization.

3.2 MiDaS as a Depth Prior

In this project, MiDaS is not treated as the final answer. Instead, it is treated as a strong depth prior. The raw output contains the overall scene geometry, but local boundary details remain imperfect. That makes MiDaS ideal as a base model for correction and enhancement.

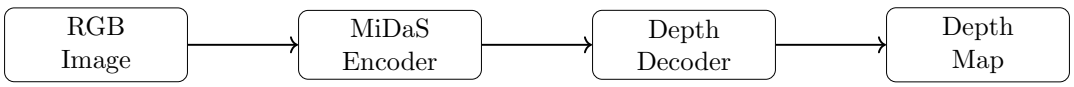

Figure 1: Simplified baseline MiDaS architecture. The input RGB image is encoded into multi-scale features and decoded into a dense depth map.

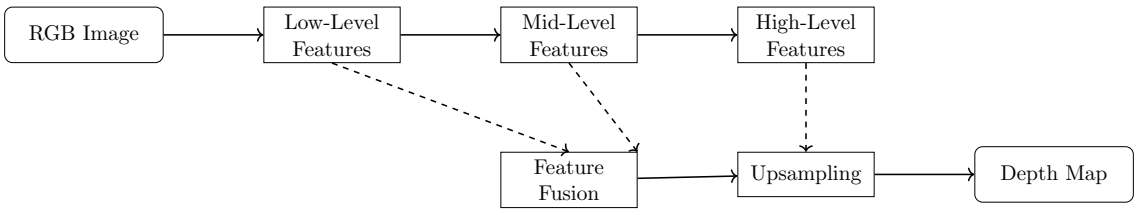

Figure 2: Research-style MiDaS architecture. The encoder extracts multi-scale features, which are fused and upsampled in the decoder to produce a dense depth map.

4 Proposed Edge-Guided Residual Depth Correction Framework

The key idea of this project is to extend the baseline MiDaS output into a stronger enhancement pipeline. Instead of stopping at post-processing, the work introduces a residual correction module that learns to improve depth boundaries and structure.

4.1 Overall System Flow

The full pipeline contains four major stages:

1. Generate raw depth using MiDaS.
2. Refine the raw depth using edge-aware guided filtering.
3. Use the guided-refined depth as a pseudo-target.
4. Train a residual network to learn the correction from the raw MiDaS prediction.

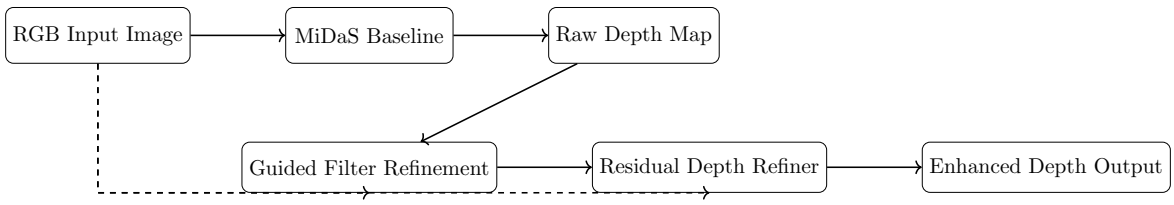

Figure 3: Overall enhancement pipeline. The RGB image first produces a raw MiDaS depth map, which is then refined using guided filtering and a learnable residual correction module.

4.2 Classical Edge-Aware Refinement

The first stage of enhancement is a classical edge-aware filtering step. This produces a refined depth map that is visually cleaner than the raw MiDaS output. The RGB image is used as guidance so that edges in the image are preserved during smoothing.

4.3 Learnable Residual Correction

After guided filtering, a small convolutional network learns to predict a residual correction:

$$D_{\text{final}} = D_{\text{MiDaS}} + \Delta D.$$

The correction network does not reconstruct the depth from scratch. Instead, it learns the missing boundary-sensitive details. This residual design is especially effective because the baseline MiDaS prediction already contains strong global structure.

4.4 Why Residual Correction is Useful

Residual correction is useful for three reasons:

- It reduces the learning burden of the network.

- It stabilizes training because the default output is close to the baseline.
- It focuses the network on local improvements instead of global re-learning.

5 Mathematical Formulation

5.1 Baseline Depth Prediction

Let the input RGB image be denoted by $I \in \mathbb{R}^{H \times W \times 3}$. MiDaS maps this image to a baseline depth map:

$$D_{\text{raw}} = \mathcal{M}(I; \theta), \tag{1}$$

where $\mathcal{M}$ is the pretrained MiDaS network and θ denotes its fixed parameters.

5.2 Depth Normalization

The predicted depth is normalized to the range $[0, 1]$:

$$D_{\text{norm}} = \frac{D_{\text{raw}} - \min(D_{\text{raw}})}{\max(D_{\text{raw}}) - \min(D_{\text{raw}}) + \epsilon}, \tag{2}$$

where ϵ is a small constant used to prevent division by zero.

5.3 Edge Map Extraction

An edge map is extracted from the RGB input using gradient magnitude:

$$E = \|\nabla I\|, \tag{3}$$

or equivalently using Sobel or Canny-based approximations. The edge map is used as an additional guidance cue for the refiner network.

5.4 Guided Filtering Model

Guided filtering assumes that within a local region ω_k, the output is a linear transform of the guidance image:

$$q_i = a_k I_i + b_k, \quad \forall i \in \omega_k. \tag{4}$$

The coefficients are computed from the local input statistics:

$$a_k = \frac{\text{cov}_{\omega_k}(I, p)}{\sigma_k^2 + \epsilon}, \qquad b_k = \bar{p}_k - a_k \bar{I}_k, \tag{5}$$

where p is the depth input, $\bar{I}_k$ and $\bar{p}_k$ are local means, and σ_k^2 is the local variance of the guidance image. The filtered output is then:

$$D_{\text{guided}} = \text{GF}(I, D_{\text{norm}}). \tag{6}$$

5.5 Residual Depth Correction

The residual refiner predicts a correction map ΔD:

$$\Delta D = f_\phi(I, D_{\text{norm}}, E), \tag{7}$$

where f_ϕ is the learnable residual network. The final enhanced depth is:

$$D_{\text{final}} = \text{clip}(D_{\text{norm}} + \Delta D, 0, 1). \tag{8}$$

The clipping operation keeps the values in the normalized range.

5.6 Loss Function

The total loss is a weighted sum of several complementary objectives:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{depth}} + \lambda_2 \mathcal{L}_{\text{res}} + \lambda_3 \mathcal{L}_{\text{grad}} + \lambda_4 \mathcal{L}_{\text{smooth}}. \tag{9}$$

Depth Loss.

$$\mathcal{L}_{\text{depth}} = \|D_{\text{final}} - D_{\text{target}}\|_1. \tag{10}$$

Residual Loss.

$$\mathcal{L}_{\text{res}} = \|\Delta D - (D_{\text{target}} - D_{\text{norm}})\|_1. \tag{11}$$

Gradient Consistency Loss.

$$\mathcal{L}_{\text{grad}} = \|\nabla_x D_{\text{final}} - \nabla_x D_{\text{target}}\|_1 + \|\nabla_y D_{\text{final}} - \nabla_y D_{\text{target}}\|_1. \tag{12}$$

Edge-Aware Smoothness Loss.

$$\mathcal{L}_{\text{smooth}} = \sum_{i,j} \left(|D_{i,j+1} - D_{i,j}| e^{-10|\nabla_x I_{i,j}|} + |D_{i+1,j} - D_{i,j}| e^{-10|\nabla_y I_{i,j}|} \right). \tag{13}$$

The weights used in the training code are:

$$(\lambda_1, \lambda_2, \lambda_3, \lambda_4) = (0.55, 0.20, 0.15, 0.10).$$

6 Implementation Details

6.1 Project File Structure

The project is organized around the following files:

- `run.py` — runs the pretrained MiDaS model and saves raw depth maps.
- `refine_depth.py` — applies guided filtering to produce refined depth outputs.
- `depth_utils.py` — helper functions for file loading, edge extraction, normalization, and panel creation.

- `edge_refiner_model.py` — defines the residual refiner network.
- `train_refiner.py` — trains the residual model using pseudo-targets.
- `infer_refiner.py` — applies the trained model to generate enhanced depth maps.

6.2 Folder Layout

The working folder is structured as:

```
MiDaS/
 input_images/
 output/
 refined_output/
 enhanced_output/
 checkpoints/
 figures/
 run.py
 refine_depth.py
 depth_utils.py
 edge_refiner_model.py
 train_refiner.py
 infer_refiner.py
 README.md
```

6.3 Baseline Inference Command

The baseline MiDaS depth maps were generated using the pretrained model and saved in the `output/` folder. A typical command is:

```
python3 run.py --model_type dpt_large_384 --input_path input_images --output_path
    output --grayscale
```

6.4 Guided Refinement Command

The guided refinement stage is executed using:

```
python3 refine_depth.py --input_dir input_images --depth_dir output --output_dir
    refined_output
```

6.5 Training Command

The residual refiner is trained using:

```
python3 train_refiner.py \
  --image_dir input_images \
  --raw_dir output \
  --target_dir refined_output/refined \
  --save_path checkpoints/edge_refiner.pt \
  --epochs 30 \
  --batch_size 4
```

6.6 Inference Command

After training, the enhanced model is applied with:

```
python3 infer_refiner.py \
  --image_dir input_images \
  --raw_dir output \
  --checkpoint checkpoints/edge_refiner.pt \
  --out_dir enhanced_output
```

6.7 Training Hyperparameters

Parameter	Value
Input image size	256×256
Refiner base channels	32
Residual scale	0.25
Optimizer	AdamW
Learning rate	10^{-3}
Weight decay	10^{-4}
Batch size	4
Number of epochs	30
Loss weights	$(0.55, 0.20, 0.15, 0.10)$

Table 1: Training and implementation hyperparameters.

7 Experimental Setup

7.1 Input Images

The project uses a small collection of indoor and outdoor RGB images. Indoor scenes typically include furniture, walls, tables, chairs, and other structured objects, while outdoor scenes include roads, buildings, and broader depth variations. This mixture is useful because the enhancement system must work across different scene layouts.

7.2 Training Targets

The residual refiner is trained using pseudo-targets generated by the guided-filter stage. This is a practical choice because the guided output is already cleaner than the raw MiDaS result and therefore acts as a better target for learning boundary-aware corrections.

7.3 Evaluation Style

The evaluation is primarily qualitative. The project focuses on whether the enhanced depth map:

- sharpens object boundaries,
- reduces noise in flat regions,

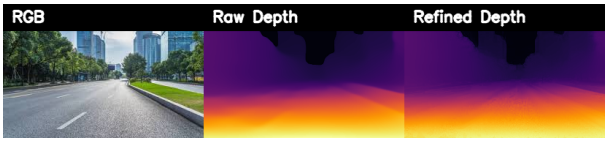

Figure 5: Enter Caption

- aligns better with RGB edges,
- and preserves the global scene depth structure.

8 Results and Analysis

The comparison outputs are the most important evidence for the project. For each image, the result panel shows the original RGB input, the raw MiDaS depth, and the enhanced output. The enhanced depth maps are expected to be visually cleaner and structurally more coherent than the baseline.

8.1 Typical Observations

The following improvements are generally visible:

- sharper object outlines,
- clearer separation between foreground and background,
- smoother depth values in flat regions,
- fewer spurious speckles or noisy patches,
- and better correspondence between image edges and depth transitions.

8.2 Figure Gallery

Place your comparison images in the `figures/` folder and include them here. Use filenames without spaces for easier compilation, such as `download_1_compare.png`, `download_2_compare.png`, and so on.

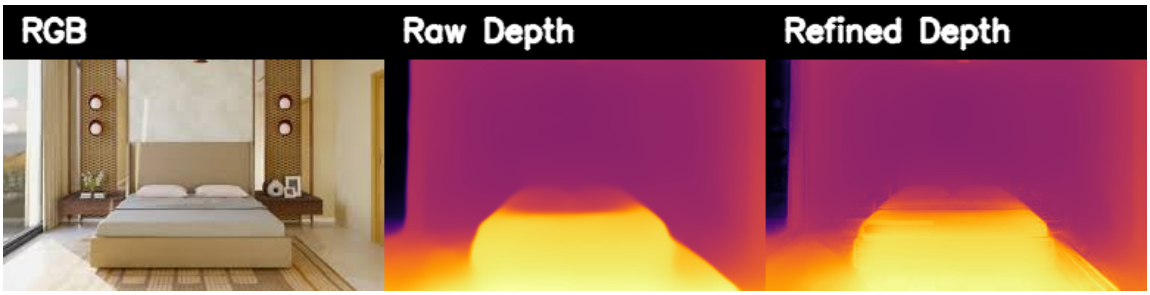

Figure 4: Representative comparison panel showing RGB input, raw MiDaS depth, and enhanced depth output.

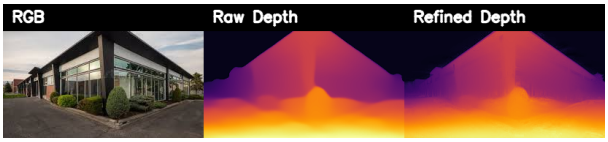

Figure 6: Enter Caption

8.3 Detailed Qualitative Discussion

The enhanced output tends to preserve the global structure estimated by MiDaS while improving the local details that matter most for interpretation. The correction is strongest around:

- chair edges,
- table borders,
- walls and room corners,
- building facades,
- road boundaries,
- and object silhouettes.

These are exactly the regions where boundary errors are most noticeable in the raw depth map. By learning a residual correction over the guided-filter output, the refiner captures these local corrections in a data-driven manner.

8.4 Failure Cases

No enhancement method is perfect. The current system may still fail in the following situations:

- low-contrast boundaries in the RGB image,
- highly reflective or transparent surfaces,
- complex thin structures,
- very noisy or low-quality input images,
- and scenes where the baseline MiDaS prediction is already very poor.

These cases are important because they show the limits of residual correction when the underlying baseline prior is weak.

9 Discussion

The project demonstrates that a pretrained depth estimator can be extended into a stronger model enhancement pipeline using a combination of classical and learnable components. The raw MiDaS output provides a strong initial estimate, but local quality still matters. The guided filter acts as a stable structural refinement step, while the residual network learns to improve the refinement further.

A key advantage of the residual formulation is that it does not force the network to relearn the entire depth estimation problem. Instead, the model only learns the correction to a strong prior. This is both more efficient and more stable. It also makes the entire system more interpretable: the enhancement module can be understood as a boundary-aware correction layer that adjusts local details while leaving the global scene geometry mostly unchanged.

Another important aspect is that the project shows a practical path from baseline inference to model enhancement. The baseline MiDaS output is not the end of the pipeline; it becomes the starting point for further structural improvement. This is a meaningful extension because it transforms a depth prediction model into a depth enhancement framework.

10 What Has Been Done in This Project

This project implements a complete enhancement pipeline:

- raw depth estimation using MiDaS,
- edge-aware guided filtering to create better pseudo-targets,
- residual network design for learning boundary-aware corrections,
- training and inference scripts for the enhancement model,
- and comparison output generation for visual inspection.

This progression makes the project more than a standard application. It becomes a model improvement framework built on top of a strong pretrained depth estimator.

11 Future Work

There are several clear directions for improving the project further:

- train on real RGB-D datasets such as NYU Depth V2 or KITTI,
- evaluate using quantitative metrics such as L1, RMSE, and edge F-score,
- integrate multi-scale features from the MiDaS encoder,
- add attention mechanisms to emphasize boundaries,
- and test lighter architectures for real-time deployment.

These directions would make the enhancement model even stronger and more research-oriented.

12 Conclusion

This project developed an edge-guided residual depth correction framework on top of MiDaS. The baseline model provides a strong relative depth prior, but the raw predictions still contain boundary blur and local artifacts. To improve this, the project first applies guided filtering to build a cleaner reference and then trains a residual network to predict the missing correction.

The final enhanced output preserves the global scene layout while improving local structure, edge clarity, and perceptual quality.

The main value of this project is that it does not treat MiDaS as the final answer. Instead, it turns MiDaS into the foundation of a larger enhancement system. That extension is what makes the work research-like: the model is not simply used, but improved through a structured, edge-aware correction pipeline.

A Commands Used

A.1 Install Dependencies

```
python3 -m pip install --upgrade pip
python3 -m pip install torch torchvision timm imutils numpy opencv-python opencv-
    contrib-python
```

A.2 MiDaS Baseline

```
python3 run.py --model_type dpt_large_384 --input_path input_images --output_path
    output --grayscale
```

A.3 Guided Refinement

```
python3 refine_depth.py --input_dir input_images --depth_dir output --output_dir
    refined_output
```

A.4 Training the Residual Refiner

```
python3 train_refiner.py \
  --image_dir input_images \
  --raw_dir output \
  --target_dir refined_output/refined \
  --save_path checkpoints/edge_refiner.pt \
  --epochs 30 \
  --batch_size 4
```

A.5 Running the Enhanced Model

```
python3 infer_refiner.py \
  --image_dir input_images \
  --raw_dir output \
  --checkpoint checkpoints/edge_refiner.pt \
  --out_dir enhanced_output
```

B Suggested Figure Naming Convention

For clean compilation, rename all comparison figures to underscore-based names such as:

- `download_1_compare.png`
- `download_2_compare.png`
- `download_3_compare.png`
- `download_4_compare.png`

Then place them in the `figures/` folder and update the `\includegraphics` commands accordingly.

C References

References

[1] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[2] R. Birkl, D. Wofk, and M. Müller, "MiDaS v3.1 – A Model Zoo for Robust Monocular Relative Depth Estimation," arXiv:2307.14460, 2023.

[3] K. He, J. Sun, and X. Tang, "Guided Image Filtering," in *Computer Vision – ECCV 2010*, Lecture Notes in Computer Science, vol. 6311, pp. 1–14, 2010.

[4] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," *CVPR*, 2017.

[5] OpenCV Documentation, "ximgproc and edge-preserving filtering utilities," https://docs.opencv.org/.